

Markov Decision Processes

CHAPTER 17

Slides have been adopted from Dan Klein and Dieter Fox

Example: Grid World

❑ A maze-like problem

- The agent lives in a grid
- Walls block the agent's path

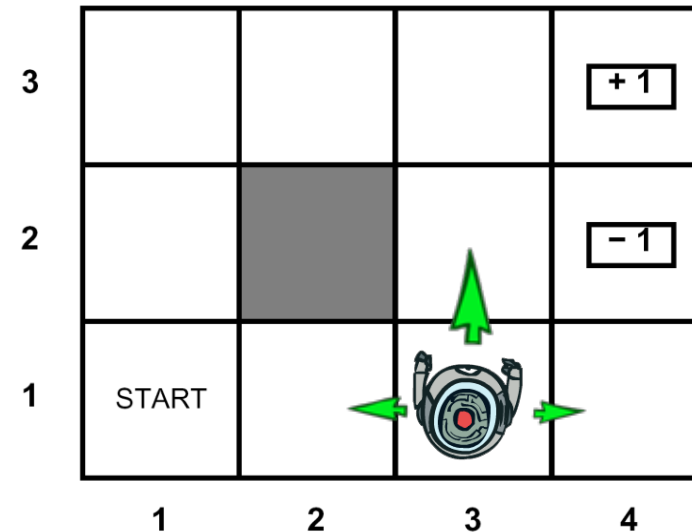
❑ Noisy movement: actions do not always go as planned

- 80% of the time, the action North takes the agent North (if there is no wall there)
- 10% of the time, North takes the agent West; 10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put

❑ The agent receives rewards each time step

- Small “living” reward each step (can be negative)
- Big rewards come at the end (good or bad)

❑ Goal: maximize sum of rewards



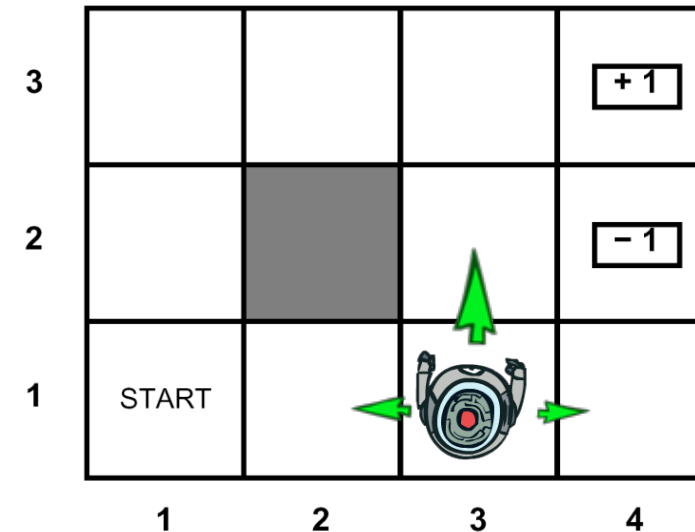
Markov Decision Processes -Elements

□ An MDP is defined by:

- A set of states s - S
- A set of actions a - A
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A start state
- Maybe a terminal state

□ MDPs are non-deterministic search problems

- One way to solve them is with expectimax search



What is Markov about MDPs?

- ❑ “Markov” generally means that given the present state, the future and the past are independent
- ❑ For Markov decision processes, “Markov” means action outcomes depend only on the current state

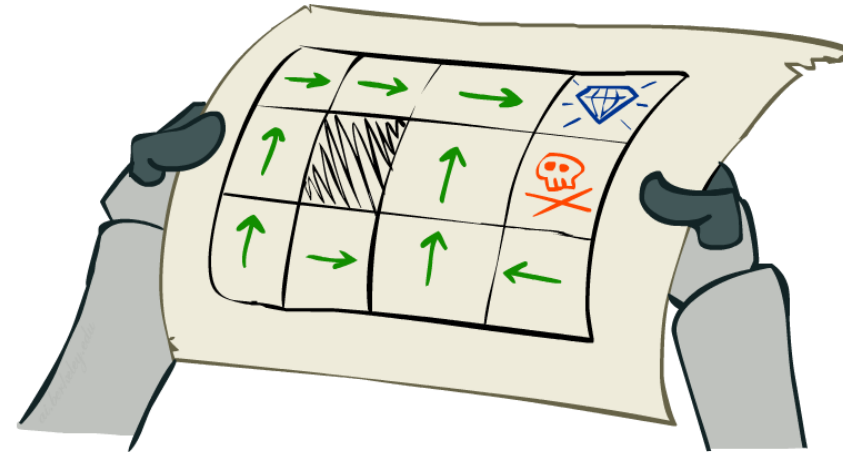
$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, \dots, S_0 = s_0) = \\ P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$



- ❑ This is just like search, where the successor function could only depend on the current state (not the history)

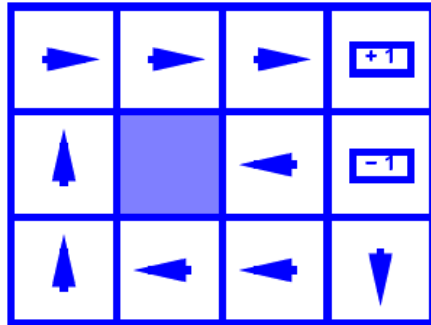
Policies

- ❑ In deterministic single-agent search problems, we wanted an optimal plan, or sequence of actions, from start to a goal
- ❑ For MDPs, we want an optimal policy $\pi^*: S \rightarrow A$
 - A policy gives an action for each state
 - An optimal policy is one that maximizes expected utility if followed
 - An explicit policy defines a reflex agent
- ❑ Expectimax didn't compute entire policies
 - It computed the action for a single state only

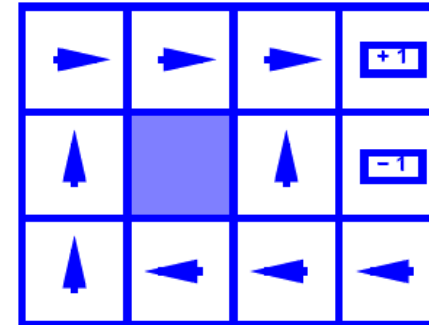


Optimal policy when $R(s, a, s') = -0.03$
for all non-terminals s

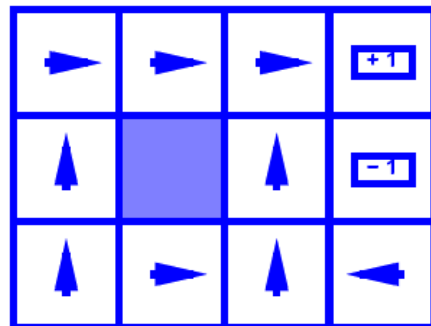
Optimal Policies



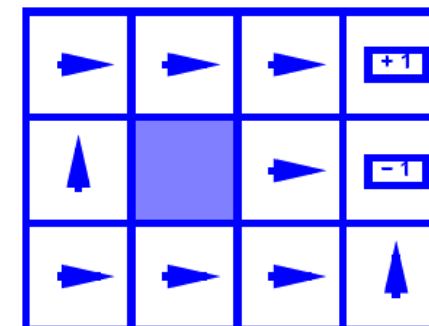
$$R(s) = -0.01$$



$$R(s) = -0.03$$



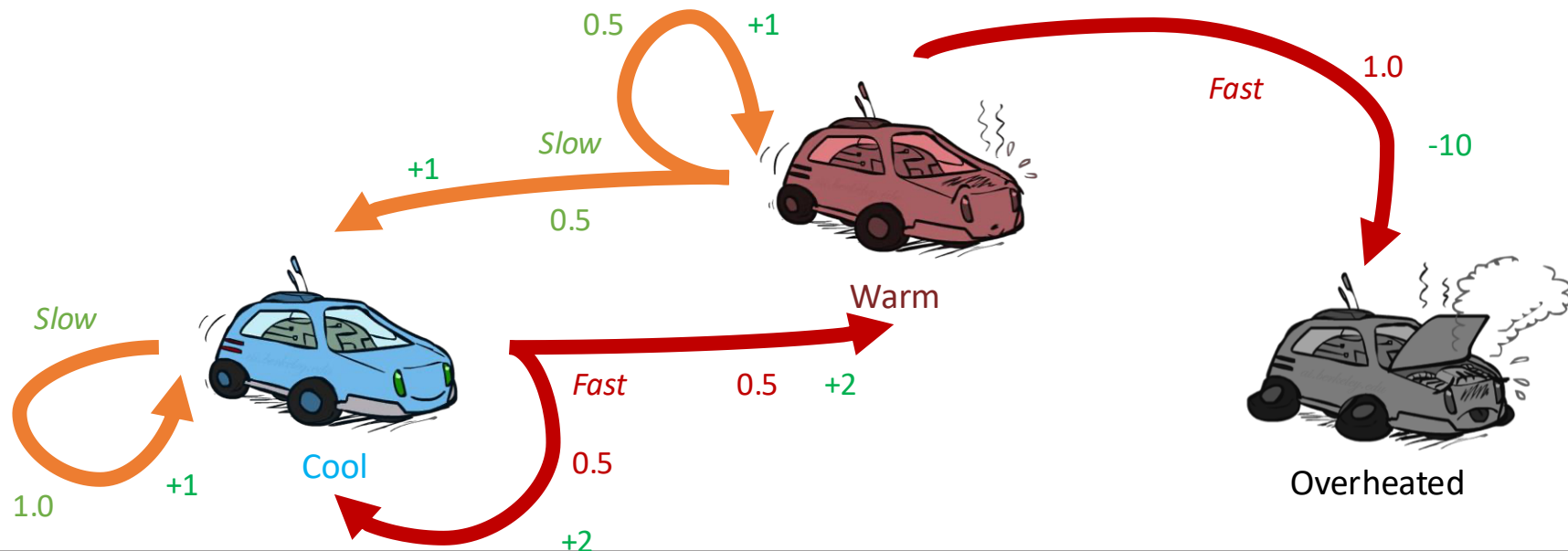
$$R(s) = -0.4$$



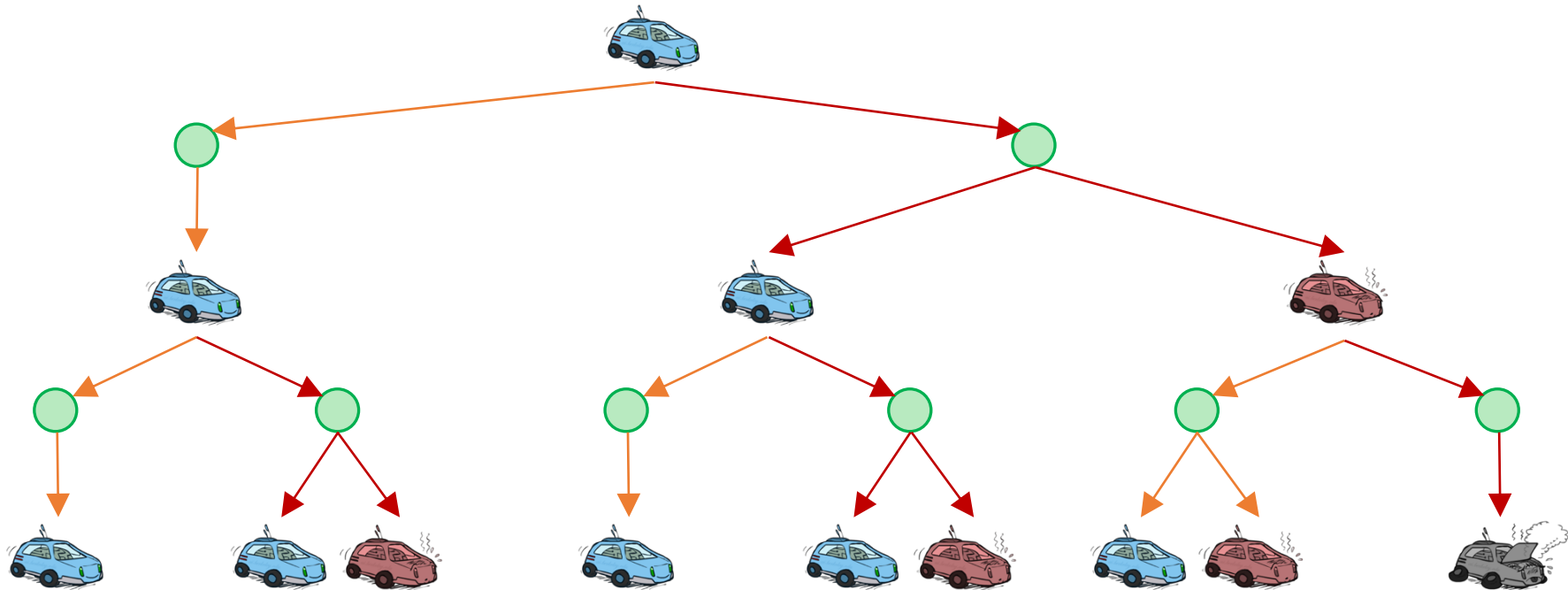
$$R(s) = -2.0$$

Example: Racing

- ❑ A robot car wants to travel far, quickly
- ❑ Three states: Cool, Warm, Overheated
- ❑ Two actions: Slow, Fast
- ❑ Going faster gets double reward

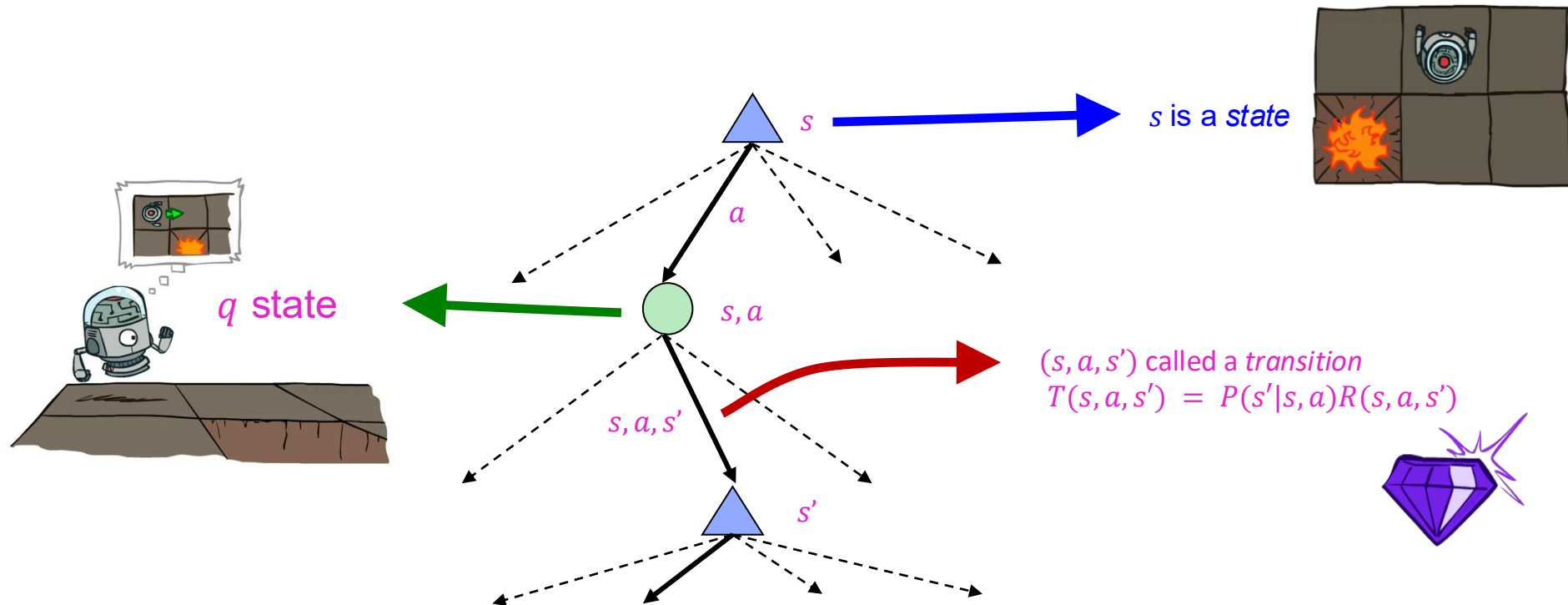


Racing Search Tree



MDP Search Trees

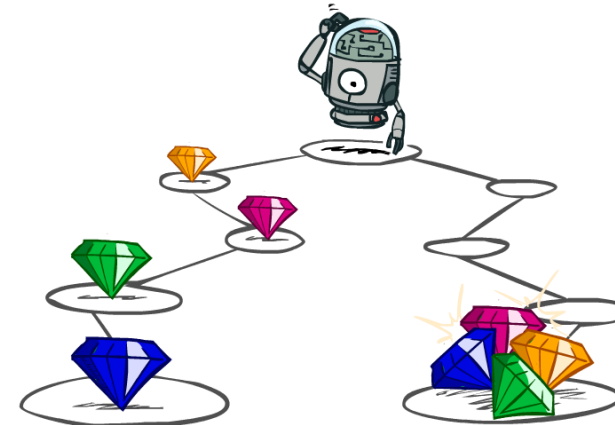
- Each MDP state projects an expectimax-like search tree



Utilities of Sequences/ over Time

□ What preferences should an agent have over reward sequences?

- More or less? $[1, 2, 3]$ or $[2, 3, 4]$
- Now or later? $[0, 0, 1]$ or $[1, 0, 0]$



Discounting

- ❑ It is reasonable to maximize the sum of rewards
- ❑ It is also reasonable to prefer rewards now to rewards later
- ❑ One solution: values of rewards decay exponentially



1

Worth Now



γ

Worth Next Step



γ^2

Worth In Two Steps

Discounting (2)

How to discount?

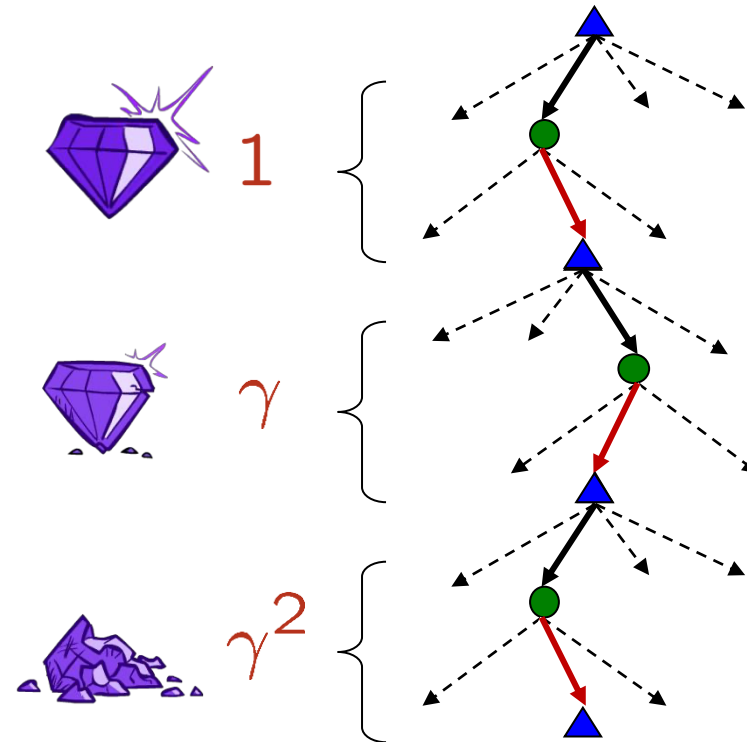
- Each time we descend a level, we multiply in the discount once

Why discount?

- Sooner rewards probably do have higher utility than later rewards
- Also helps our algorithms converge

Example: discount of 0.5

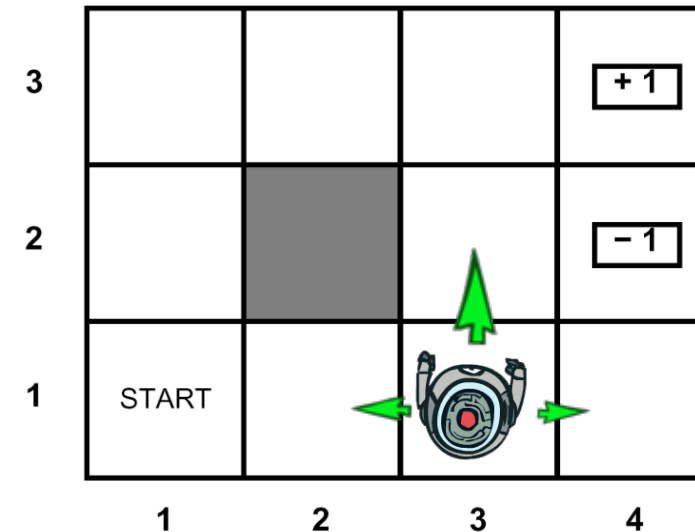
- $U([1,2,3]) = 1*1 + 0.5*2 + 0.25*3$
- $U([1,2,3]) < U([3,2,1])$



Recap - Markov Decision Processes - Elements

□ An MDP is defined by:

- A set of states s - S
- A set of actions a - A
- A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s' | s, a)$
 - Also called the model or the dynamics
- A reward function $R(s, a, s')$
 - Sometimes just $R(s)$ or $R(s')$
- A start state
- Maybe a terminal state



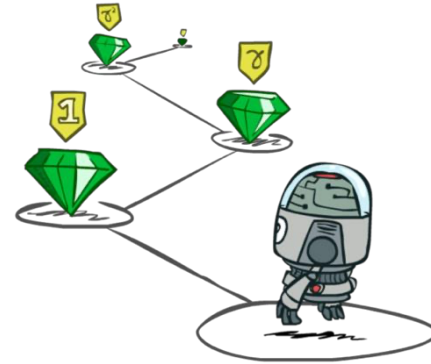
□ Policies

□ Stationary Preferences

Stationary Preferences

□ Theorem: if we assume stationary preferences:

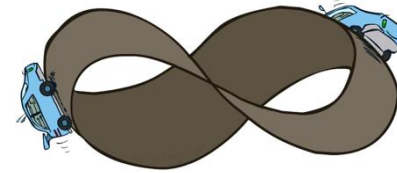
$$\begin{aligned} [a_1, a_2, \dots] &> [b_1, b_2, \dots] \\ \Updownarrow \\ [r, a_1, a_2, \dots] &> [r, b_1, b_2, \dots] \end{aligned}$$



□ Then: there are only two ways to define utilities

- Additive utility: $U([r_0, r_1, \dots]) = r_0 + r_1 + r_2 + \dots$
- Discounted utility: $U([r_0, r_1, \dots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

Utilities Over Time



❑ Problem: What if the game lasts forever? Do we get infinite rewards?

❑ Solutions:

- Finite horizon: (similar to depth-limited search)

- Terminate episodes after a fixed T steps
- Gives nonstationary policies (π depends on time left)

- Discounting: use $0 < \gamma < 1$

$$U([r_0, r_1, \dots]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq R_m / (1 - \gamma)$$

- Smaller γ means smaller “horizon” – shorter term focus
- Absorbing state: guarantee that for every policy, a terminal state will eventually be reached (like “overheated” for racing)

Discounting (3)

□ Given:

10				1
a	b	c	d	e

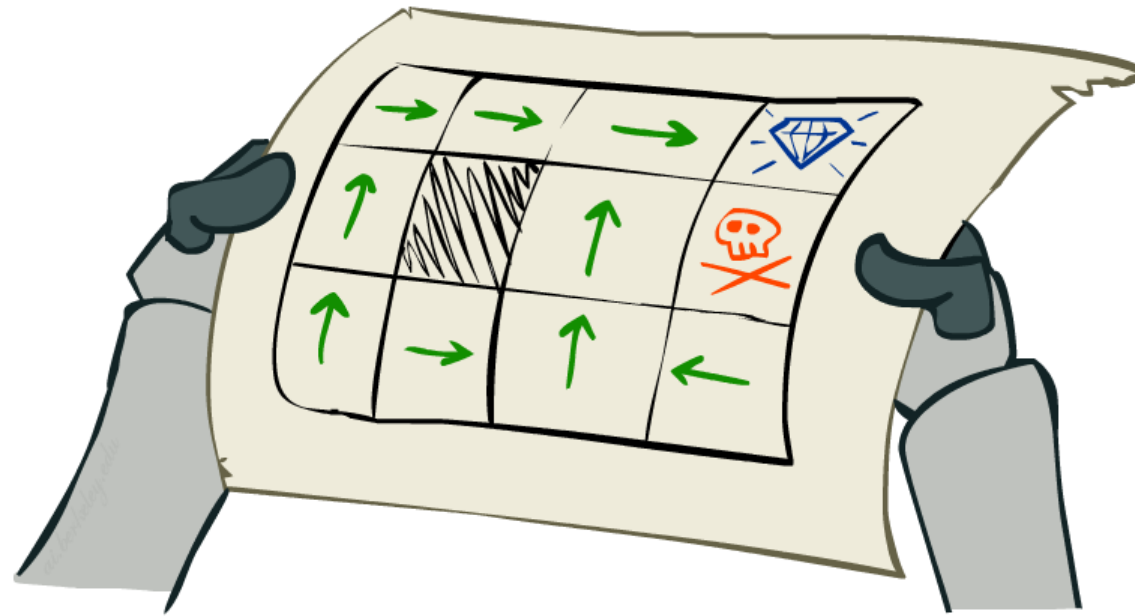
- Actions: East, West, and Exit (only available in exit states a, e)
- Transitions: deterministic

□ Q 1: For $\gamma = 1$, what is the optimal policy?

□ Q2: For $\gamma = 0.1$, what is the optimal policy?

□ Q3: For which γ are West and East equally good when in state d?

Solving MDPs



Optimal Quantities

□ The value (utility) of a state s :

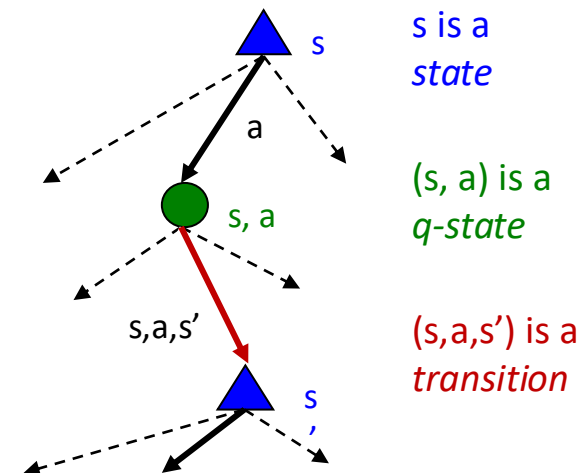
- $V^*(s)$ = expected utility starting in s and acting optimally

□ The value (utility) of a q-state (s, a) :

- $Q^*(s, a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally

□ The optimal policy:

- $\pi^*(s)$ = optimal action from state s



Snapshot of Gridworld V Values



Noise = 0.2
Discount = 0.9
Living reward = 0

Snapshot of Gridworld Q Values



Noise = 0.2
Discount = 0.9
Living reward = 0

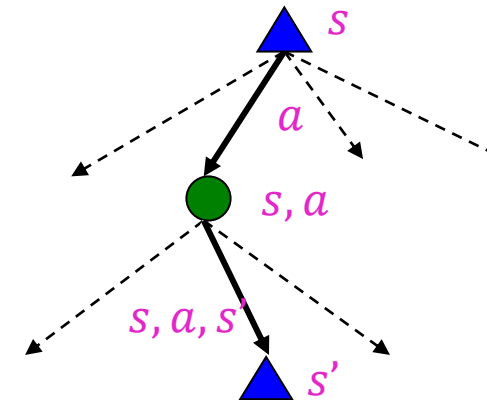
Values of States

□ Fundamental operation: compute the (expectimax) value of a state

- Expected utility under optimal action
- Average sum of (discounted) rewards

□ Recursive definition of value –

- Bellman Equations:
- $V^*(s) = \max_a Q^*(s, a)$
- $Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$
- $V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$



Racing Search Tree

❑ We're doing way too much work with expectimax!

- Problem: States are repeated

 - Idea: Only compute needed quantities once

- Problem: Tree goes on forever

 - Idea: Do a depth-limited computation, but with increasing depths until change is small

❑ Note: deep parts of the tree eventually don't matter if $\gamma < 1$

